

PagedAttention is an attention algorithm for efficient serving of large language models (LLMs). It was introduced in 2023 by Woosuk Kwon and colleagues in the paper Efficient Memory Management for Large Language Model Serving with PagedAttention, alongside the vLLM serving engine. The method stores the key-value cache used during autoregressive decoding in fixed-size blocks that can be mapped to non-contiguous physical memory, borrowing ideas from virtual memory, paging, and operating system design.

Background

In transformer inference, the key-value cache grows with sequence length and the number of concurrent requests. Kwon et al. argued that earlier serving systems typically reserved contiguous cache regions in advance, which caused reserved space, internal fragmentation, and external fragmentation. In their experiments, the paper reported that the effective memory utilization of previous systems could fall as low as 20.4%.

Description

PagedAttention partitions the cache of each sequence into fixed-size KV blocks. A request's cache is represented as a sequence of logical blocks, while a block table maps those logical blocks to physical GPU-memory blocks. As a result, neighboring logical blocks do not need to be contiguous in physical memory, and new blocks can be allocated on demand as generation proceeds.

The design also makes it easier to share cache state across related decoding paths. In vLLM, physical blocks can be reference-counted and shared among requests or branches, with block-granularity copy-on-write used when a shared block must be modified. The original paper applied this design to parallel sampling, beam search, and prompts with shared prefixes.

Mathematical formulation

For a query token

$$i$$

$${\displaystyle i}$$

in causal self-attention, the standard attention output can be written as

$$a$$

$$i$$

$$j$$

$$=$$

exp

(

q

i

T

k

j

/

d

)

Σ

t

=

1

i

exp

(

q

i

T

k

t

/

d

)

,

o

i

=

Σ

j

=

1

i

a

i

j

v

j

$$\{ \displaystyle a_{ij} = \frac{\exp(\mathbf{q}_i^{\text{top}} \mathbf{k}_j / \sqrt{d})}{\sum_{t=1}^i \exp(\mathbf{q}_t^{\text{top}} \mathbf{k}_j / \sqrt{d})} \}, \mathbf{o}_i = \sum_{j=1}^i a_{ij} \mathbf{v}_j \}$$

where

q

i

$$\{\displaystyle \mathbf{q}_{\{i\}}$$

,

k

j

$$\{\displaystyle \mathbf{k}_{\{j\}}$$

, and

$\mathbf{v}$

j

$\{\mathbf{v}_{\{j\}}\}$

are the query, key, and value vectors, and

d

d

is the attention dimension.

If the cache is partitioned into blocks of size

B

$${\displaystyle B}$$

, the key and value blocks may be written as

K

j

=

(

k

(

j

-

1

)

B

+

1

,

...

,

k

j

B

)

,

V

j

=

(

v

(

j

-

1

)

B

+

1

,

...

,

V

j

B

)

$$\{\displaystyle \mathbf{K}_{\{j\}}=(\mathbf{k}_{\{(j-1)B+1\}},\ldots,\mathbf{k}_{\{jB\}}),\;\mathbf{V}_{\{j\}}=(\mathbf{v}_{\{(j-1)B+1\}},\ldots,\mathbf{v}_{\{jB\}})\}$$

PagedAttention then performs the computation blockwise:

A

i

j

=

exp

(

q

i

T

K

j

/

d

)

Σ

t

=

1

Γ

i

/

B

1

exp

(

q

i

T

K

t

/

d

)

,

o

i

=

Σ

j

=

1

[

i

/

B

]

V

j

A

i

j

T

$$\{\displaystyle \mathbf{A}_{ij}=\frac{\exp(\mathbf{q}_{i}^{\text{top}})\mathbf{K}_{j}}{\sqrt{d}}\}\{\sum_{t=1}^{\lceil i/B\rceil}\exp(\mathbf{q}_{i}^{\text{top}})\mathbf{K}_{t}/\sqrt{d}\}\},\backslash\mathbf{o}_{i}=\sum_{j=1}^{\lceil i/B\rceil}\mathbf{V}_{j}\mathbf{A}_{ij}^{\text{top}}\}$$

where

A

i

j

$$\mathbf{A}_{ij}$$

is the vector of attention scores for the

j

$$j$$

-th KV block. In the formulation given by Kwon et al., this preserves the causal attention calculation while allowing the key and value blocks to reside in non-contiguous physical memory.

Performance and use

The vLLM paper reported that, on its evaluated workloads, the use of PagedAttention and the associated memory-management design improved serving throughput by 2–4× over the

compared baselines, including FasterTransformer and Orca, while preserving model outputs. In experiments on OPT-13B with the Alpaca trace, the paper also reported memory savings of 6.1–9.8% for parallel sampling and 37.6–55.2% for beam search through KV-block sharing. A 2024 survey of LLM serving systems described PagedAttention as having become an industry norm in LLM serving frameworks, citing support in TGI, vLLM, and TensorRT-LLM.

Limitations and alternatives

Subsequent work has described trade-offs in the approach. The 2025 vAttention paper argued that PagedAttention requires attention kernels to be rewritten to support paging and increases software complexity, portability issues, redundancy, and execution overhead, proposing instead a memory manager that keeps the cache contiguous in virtual memory while relying on demand paging for physical allocation.

vAttention

Unlike PagedAttention, vAttention does not introduce a different attention rule; it retains the standard attention computation

Attention

(

q

i

,

K

,

V

)

=

softmax

(

q

i

K

T

s

c

a

l

e

)

V

.

$$\text{Attention}(q_i, K, V) = \text{softmax} \left(\frac{q_i K^{\text{top}}}{\text{scale}} \right) V.$$

In the notation of Prabhu et al., the key and value tensors for a request seen so far are

K

,

V

∈

R

L

,

×

(

H

×

D

)

$$\{K,V\in \mathbb{R} ^{L\times (H\times D)}\}$$

,

where

L

,

$$L$$

is the context length seen so far,

H

$$H$$

is the number of KV heads on a worker, and

D

$\displaystyle D$

is the dimension of each KV head.

In systems prior to PagedAttention, the K cache (or V cache) at each layer of a worker is typically allocated as a 4D tensor of shape

[
B
,
L
,
H
,
D
]
,

$\displaystyle [B,L,H,D],$

where

$$B$$

$${\displaystyle B}$$

is batch size and

$$L$$

$${\displaystyle L}$$

is the maximum context length supported by the model.

vAttention preserves this contiguous virtual-memory view while deferring physical-memory allocation to runtime. A serving framework maintains separate K and V tensors for each layer, so vAttention reserves

2

N

$${\displaystyle 2N}$$

virtual-memory buffers on a worker, where

N

$${\displaystyle N}$$

is the number of layers managed by that worker.

The maximum size of one virtual-memory buffer is

B

S

=

B

×

S

,

$$BS=B\times S,$$

where

S

$$S$$

is the maximum size of a single request's per-layer K cache (or V cache) on a worker. The paper defines

S

=

L

×

H

×

D

×

P

,

$$S=L\times H\times D\times P,$$

where

P

$$P$$

is the number of bytes needed to store one element.

In this formulation, vAttention keeps the KV cache contiguous in virtual memory and relies on demand paging for physical allocation, rather than modifying the attention kernel to operate over non-contiguous KV-cache blocks.

See also

AlphaDev and AlphaEvolve — AI systems by Google DeepMind for discovering and optimizing computer science algorithms

Attention mechanism

Beam search

Cache replacement policies

List of artificial intelligence algorithms

== References ==